5. css自定义属性 / 用户行为与体验增强

5.1 css自定义属性 (css变量)

css自定义属性，即css变量，使用方式很简单，`--` + 变量名，声明变量； var() 函数调用变量，调用的时候也需要带上 `--` ;css的变量名限制很少，可以用数字中文，不要用奇奇怪怪的特殊字符就好，如`$ [ ] ^ ( ) %`

CSS

```
:root {
    --primary-color: deepskyblue;
}
button {
    background-color: var(--primary-color);
}
```

var()

CSS

```
var( <custom-property-name> [, <declaration-value> ]? )
```

用于获取变量值，与之前讲的 env() 是类似的，第一个参数传变量名，后面的参数传可选的备选值(可能有多个)，当第一个参数的计算是保证无效的值(Guaranteed-Invalid Values)时，后面的参数依次生效，我们先来看个例子👀，大家可以猜看看 `background-color` 最后生效的值是什么

CSS

```
body {
    --color: 20px;
    background-color: deeppink;
    background-color: var(--color, deepskyblue);
}
```

A declaration can be ***invalid at computed-value time*** if it contains a 'var()' that references a custom property with the guaranteed-invalid value, as explained above, or if it uses a valid custom property, but the property value, after substituting its 'var()' functions, is invalid. When this happens, the computed value is one of the following depending on the property's type:

↳ **The property is a non-registered custom property**
↳ **The property is a registered custom property with universal syntax**
The computed value is the guaranteed-invalid value.

↳ **Otherwise**

Either the property's inherited value or its initial value depending on whether the property is inherited or not, respectively, as if the property's value had been specified as the 'unset' keyword.

最后生效的值其实是transparent，我们可以在上面这段释意看到，当var()调用的自定义属性未声明 或 其值是通用语法(我的理解是全局关键字)时，第一个参数的计算值就是 guaranteed-invalid values，此时var()会取后备值，其余var()取无效值的结果类似于 unset，即能inherit的属性inherit，不能inherit的属性initial；这里只是效果类似于 unset，并不是直接将属性值设置为unset；

而background-color是不可继承的，因此取其initial值transparent；

还有一点需要注意的是，只有通过var()访问无效语法时才会出现类似unset的效果，而直接写background-color: 20px 是会使这一条语句直接失效的；

我们可以运用这个特性，通过只改变 👆 一个变量，使得多个属性值变化，精简代码

https://codepen.io/dqqbl/pen/ZErKyPQ

很好 没什么用的知识又增加了👌

css自定义属性需要主要的点还有：

- 变量有作用域，只对当前元素及后代元素生效
- 可以嵌套使用，但不能相互引用或自调用
- 不能在媒体查询的条件中使用，如 `@media (max-width: var(--maxWidth)) {}`，不合法
- 不能作为伪元素content的值直接使用，如 `div::after { content: var(--color) }` 无效
- 通过js操作css变量的方法
  - 取值 `window.getComputedStyle(box).getPropertyValue('--color')`
  - 赋值 `box.style.setProperty('--color', 'deepskyblue');`
- css自定义属性支持任何类型的属性值，借助这一点我们甚至可以创建自己的css语法

  比如，我们可以通过rgba实现数值颜色的透明效果，但却没有办法实现颜色关键字的透明效果，比如

  `color: rgba(255, 192, 203, .5)` ㊐

`color: pink .5` 🤦‍♀️

而借助css自定义属性和js，我们就可以实现一个类似 keyword(pink, .5) 的方法来实现颜色关键字的透明

https://codepen.io/dqqbl/pen/ExQmoam

主要的思路就是，获取页面中所有包含keyword()函数的自定义属性；遍历所有DOM，如果设置了对应的自定义属性，则将keyword()函数语法转换成浏览器能够识别的rgba()函数语法

5.2 用户行为与体验增强

5.2.1 滚动行为相关

scroll-behavior

scroll-behavior是一个交互效果渐进增强的CSS属性，用于控制页面的滚动效果，auto为初始值，smooth可以使页面的滚动效果变得更平滑，例；这种渐进式的效果可以尽量使用；

CSS

```
scroll-behavior: auto | smooth
```

overscroll-behavior

在默认情况下，局部滚动的滚动条滚动到底部边缘再继续滚动的时候，外部容器滚动条会继续跟着滚动。但是，有时候我们希望局部滚动的滚动条滚动到底部之后，滚动行为就停止；overscroll-behavior属性就用于控制DOM元素滚动到边缘时的行为；取一个值时xy轴取值相同，取两个值时分别表示x和y轴

CSS

```
overscroll-behavior: [ contain | none | auto ]{1,2}
```

- auto: 默认值，表现为我们默认看到的滚动行为，即滚动条滚动到边缘后继续滚动外部的可滚动容器
- contain: 默认的滚动溢出行为只会表现在当前元素的内部(例 如“反弹”效果或刷新)，不会对相邻的滚动区域进行滚动。即滚动条滚动到边缘后不会滚动外部的可滚动容器
- none: 相邻的滚动区域不会发生滚动，并且会阻止默认的滚动溢出行为

contain和none的区别在于移动端滚动溢出时的“反弹”效果，取none时无该默认效果

(safari 不支持该属性)

overflow-anchor

滚动锚定，该属性用于解决视口上方内容突然出现导致的页面跳动问题，比如在某个图片很多的页面中(如漫画网站)，如果上方的图片加载过慢，那么当前视口内已经加载的内容就会被推下去，导致不好的体验；

```
CSS
overflow-anchor: auto | none
```

初始值是auto，表示浏览器自己决定是否锚定视口，通常情况下是锚定的，即内容变动不影响当前视口；但是有时我们需要的是点击一个按钮，然后内容出现，这种情况下内容的优先级更高，不需要锚定效果，可以将值设置为none

（safari 及移动端支持不好）

CSS Scroll Snap

CSS Scroll Snap是CSS中一个独立的模块，可以让网页容器滚动停止的时候，自动平滑定位到指定元素的指定位置，例，过去需要通过js计算实现的效果现在可以通过css轻松实现😉；该模块由多个属性组成，这里只介绍有可用性的

- scroll-snap-type

```
CSS
scroll-snap-type: y mandatory;
```

该属性设置在容器上，取值由两部分构成，一部分用于确定生效的滚动方向

  - none: 默认值，表示滚动时忽略捕捉点，也就是我们平时使用的滚动
  - x: 捕捉水平定位点
  - y: 捕捉垂直平定位点
  - block: 捕捉和块状元素排列一个滚动方向的定位点，默认文档流下指的就是垂直轴
  - inline: 捕捉和内联元素排列一个滚动方向的定位点，默认文档流下指的就是水平轴
  - both: 横轴、纵轴都捕捉

另一部分用于确定滚动结束后是否进行定位

  - mandatory: 表示“强制”，也就是如果存在有效的定位点位置，则滚动容器必须在滚动结束时进行定位；(亲测需要兄弟元素出现在视窗内才生效)
  - proximity: 表示“大约”，可能会定位，这个值的作用表现为让浏览器自己判断要不要定位

大多数时候，参数mandatory和proximity的效果是类似的，只有在滚动容器的窗口尺寸比子元素的尺寸还要小的时候，参数 mandatory和proximity的差异才会体现出来，例

- scroll-snap-align

```
CSS
scroll-snap-align: [ none | start | end | center ]{1,2}
```

该属性设置在子元素上，用于设置定位点位置，设置两个值时分别表示block方向和inline方向；该值的默认值是none，经我个人实践此时没有定位效果，建议设置为非none

滚动条样式

标准的滚动条样式规则仅火狐支持

```
CSS
scrollbar-width: auto | thin | none;
scrollbar-color: auto | dark | light | <color>{2};
```

而非标准的 `::webkit-scrollbar-*` 属性，仅在基于 Blink 或 WebKit 的浏览器支持(例如，Chrome、Edge、Opera、Safari、iOS 上所有的浏览器等，也就是除了火狐外的大多数浏览器😅），而且实现效果更好；咱们的孙子和播鸭里也有封装相关函数，可以自行了解下

CSS 滚动条选择器

你可以使用以下伪元素选择器去修改基于 webkit 的浏览器的滚动条样式：

- `::-webkit-scrollbar`——整个滚动条。
- `::-webkit-scrollbar-button`——滚动条上的按钮（上下箭头）。
- `::-webkit-scrollbar-thumb`——滚动条上的滚动滑块。
- `::-webkit-scrollbar-track`——滚动条轨道。
- `::-webkit-scrollbar-track-piece`——滚动条没有滑块的轨道部分。
- `::-webkit-scrollbar-corner`——当同时有垂直滚动条和水平滚动条时交汇的部分。通常是浏览器窗口的右下角。
- `::-webkit-resizer`——出现在某些元素底角的可拖动调整大小的滑块。

```
.changeContainerScrollbar(@display: block, @scrollbarSize: 7px, @scrollbarThumbBg: #c1c1c1, @scrollbarThumbRadius: 20px, @scrollbarTrackPiece: transparent) {
  &::-webkit-scrollbar {
    /* 滚动条整体样式 */
    display: @display;
    width: @scrollbarSize; /* 高宽分别对应横竖滚动条的尺寸 */
    height: @scrollbarSize;
  }
  &::-webkit-scrollbar-thumb {
    /* 滚动条自身样式 */
    background: @scrollbarThumbBg;
    border-radius: @scrollbarThumbRadius;
  }
  &::-webkit-scrollbar-track-piece {
    background: @scrollbarTrackPiece; /* 滚动条轨道样式 */
```

```
  }
}
```

5.2.2 点击行为相关

pointer-events

```CSS
pointer-events: auto | none | visiblePainted | visibleFill | visibleStroke | visible | painted | fill | stroke | all
```

pointer-events 可以控制鼠标在元素上的行为事件，除了auto和none，其余值只作用在svg元素上，表示鼠标在svg上的哪些区域时鼠标事件生效；

none声明可以让元素无视点击、鼠标悬停和拖拽等行为，于是很多开发者会将其用在按钮元素上来实现禁用效果

```CSS
button.disabled {
    opacity: .4;
    pointer-events: none;
}
```

> 我见过太多的开发者使用上面这样的CSS代码了，注意，这并不是推荐做法，有时候甚至是糟糕的

主要问题有两点：

一是该属性仅对鼠标事件生效，通过键盘tab仍可focus元素，然后enter触发点击；

二是禁用时的title属性等悬停会失效，而移动端的无障碍阅读是通过触摸触发的，如果设置了pointer-events:none，就会影响识别

因此对于控件元素按钮禁用，更推荐原生的disabled属性

```CSS
<button disabled>按钮</button>

button:disabled {
    opacity: .4;
}
```

该属性常用于视觉表现而不是控件，比如水印的覆盖层；

该属性具有继承性，容器设置none后子元素的鼠标事件都会失效，子元素可以设置auto覆盖，实现局部失效

5.2.3 拉伸行为相关

```CSS
resize: none | both | horizontal | vertical | block | inline;
```

该属性用来控制元素的拉伸行为，最常见的是textarea元素，但这个属性并不是textarea属性专属的；

https://codepen.io/dqqbl/pen/mdXWdjQ

需要注意的是：

- 该属性不支持内联元素
- 如果是块级元素，需要overflow属性的计算值不是visible
- 该属性的原理是改变元素的width和height属性，可以用min-width等属性限制尺寸
- 该属性与滚动条同源，改变滚动条尺寸也会影响拖拽按钮尺寸
- webkit内核的浏览器下，拖拽按钮可替换

```CSS
::-webkit-resizer {
    background-image: url(resize.png);
}
```

5.2.4 输入行为相关

caret-color

```
CSS
caret-color: <color>;
```

就是简单的设置光标变色

5.2.5 选择行为相关

user-select

```
CSS
user-select: auto | text | none | all | contain
```

该属性用于控制用户选择行为，最常用的就是设置为none，表示禁用图文选中，例

- text 表示文字和图片可以被选中
- all 的含义并不是所有类型的内容都可以被选中，而是元素的内容需要整体选择，包括子元素里的文字内容，可以用来模拟浏览器的全选效果
- contain 仅ie支持，别管

https://codepen.io/dqqbl/pen/wvyJawd

需要注意的是：

- 伪元素生成的内容始终无法被选中
- safari不支持 all 值

::selection

该伪元素用于改变文字被选中后的颜色和背景色，一般全局设置，可以打开开发者工具康康👀👉

```
CSS
::selection {
    background: red;
}
```

5.2.6 打印行为相关

color-adjust

用于调整打印输出效果，需要前缀支持

```
CSS
color-adjust: economy | exact;
```

- economy 默认值，表现为用户代理对元素进行样式上的调整，调整的规则由用户代理自行决定以获得更好的输出效果。例如，打印时浏览器会选择省略所有背景图像， 并调整文本颜色，以确保对比度，保证白纸上的阅读效果是最佳的
- exact 表示精确打印，不作样式调整

目前chrome支持的是非标准的 `-webkit-print-color-adjust`

break-before: page

设置了该属性的元素在打印时，始终新开一页，并在处在该页最上端，例

5.3 svg元素的css控制

基本的SVG图形效果完全可以使用CSS绘制，包括多边形、矩形(含圆角矩形)、椭圆和任意不规则形状；

```
CSS
<svg>
    <circle cx="150" cy="75" r="60" class="circle"></circle>
</svg>
/* 等同于 */
<svg>
    <circle class="circle"></circle>
</svg>

.circle {
    cx: 150px;
    cy: 75px;
    r: 60px;
}
```

使用css绘制svg的好处是：

- 增强了复用性
- 通过外部手段重置svg设置成为可能，在不修改svg元素的情况下更改svg样式，需要注意的是，优先级：内联样式 > 选择器样式 > svg元素属性；下例中 svg 的width值生效的是 300px

CSS

```
<svg style="width: 300px" width="200" height="200" version="1.1" xmlns="http://www.w3.org/2000/svg">
  <rect x="10" y="10" width="100" height="30" stroke="black" fill="transparent" stroke-width="5" />
</svg>

svg {
    width: 400px
}
```

- 相比于js，css实现svg的动画效果更方便

https://codepen.io/dqqbl/pen/LYQOMpE

控制svg的填充颜色需要用到fill属性，fill属性有继承性，在body上设置fill可以统一改变页面上svg颜色（在没有被覆盖的前提下）

CSS

```
body {
    fill: red;
}
```

控制svg的填充色要用fill，那为啥我们项目里控制svg颜色都用的color属性嘞，因为我们项目里的svg都被设置了 currentColor 关键字，我们前面的课程有学习过，这个关键字的取值是当前元素的color值，因此我们可以通过color控制svg的颜色

```
▼<svg width="1em" height="1em" fill="currentColor" aria-hidden="true" focusable="false" class> == $0
  <use xlink:href="#icon-log-out"></use>
</svg>
```

最后我们再介绍两个使用性价比较高的属性

- stroke-dasharray 可以用来控制虚线描边间隔
- stroke-dashoffset 可以用来控制描边的偏移

配合css动画可以以很少的代码实现很好的动画效果，例1，例2

https://codepen.io/dqqbl/pen/XWZzooJ